Claude Code で自作ゲームエンジンの独学用教材を作ってみた

自分に合った教材は、作れる

成果物

CPU で1ピクセル塗るところから、VRM を動かすまで

Day 73b 時点のデモ



Day 51 まで
写経の進み具合(88 枚中 54 枚目、約2.5か月)。まだ途中

88 枚
計画書の総数(Day 1〜73b)

約8.6万行
C#(Day 73b 時点)+シェーダ 約7,600行

動機

自分に合った教材が欲しかった

既存の教材（書籍、Web）とのずれ

- 言語が C++ 前提（自分は C# メイン）
- 「エンジンの使い方」か「低レイヤだけ」のどちらか
- 理論の解説だけで、実践的なコードが無い
- 欲しい内容が足りない（物理が無い、デモまで届かない）
- まとまった時間を前提にしている

自分の条件

- **目的は学習**（実用のエンジンは目指さない）
- C# / .NET、ゲーム開発 20 年
- なるべく低いレイヤから（GPU を使わない描画から）
- 1 Day ずつ自分のペースで、毎回「動くもの」ができる
- 最後は AAA 風のデモをキャラが走り回る

やったこと　この条件を Claude Code に渡して、ロードマップ・計画書・答えのコードまで作らせた

ロードマップの作成を依頼

最初の頃のプロンプト（要約）

1 超シンプルでいいので、ゲームエンジンを自作したい。グラフィックスの初歩も理解したい
→ ロードマップの骨組み（Phase 0〜5）

2 答えのコードは AI が作って、自分は写経したい。OS自作入門みたいに 1日目、2日目…と
→ reference/DayXX の形式

3 「ゲームグラフィックス特論」や手持ちの本の、どこを読めばいいか教えて
→ その日に読む資料の対応表

4 最後は AAA に近いデモを作りたい。必須とそうでないものを分けて
→ デモ必須編 / デモ推奨編

ポイント 専門用語は要らない。学びたいことを、学びたい形で、そのまま伝えた

最初にできたロードマップ

8/08 時点・全 57 Day

Day 1〜10	Day 11〜18	Day 19〜24	Day 25〜30	Day 31〜57
ソフトラスタライザ	**レンダラ**	**エンジンコア**	**ゲーム**	**デモ・教養編**
CPU で描画処理	GPU で 2D 描画	ECS とシーン	ブロック崩し 1 本	AAA 風デモと現代技術



Day 1　　帯の長さ = Day 数　　Day 57

方針を比べてから決める

ソフトウェアラスタライザ自作 / 生 OpenGL / Silk.NET / MonoGame を比較させた

Silk.NET: C# から OpenGL などの GPU の命令を呼ぶための薄いライブラリ。描画の中身は自分で書く

結論: CPU で原理を学び、生 OpenGL を一度体験したら、あとは **Silk.NET を採用**

手持ちの教科書と対応

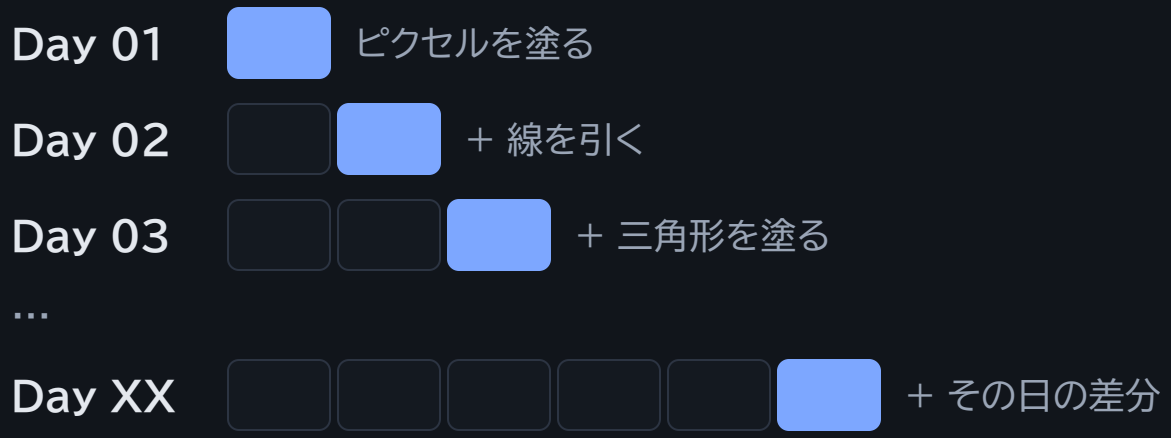
ゲームグラフィックス特論・西川本の章と Day の対応表も作らせた。「その日に読む資料」が決まる

まだシンプル

卒業制作はブロック崩し。物理エンジンもキャラの操作も無い。ここから育てていく

全体の流れ: 前の Day をコピーして、その日の分を足す

ロードマップを作る → Day 01 を作る → Day 01 をコピーして Day 02 の分を足す → Day 02 をコピーして Day 03 の分を足す → …



青 = その日に足した分。どの Day も、そこまでの完動品

お手本:『30日でできる! OS自作入門』

1日目のソースコード、2日目のソースコード、…、30日目の完成品、と毎日の完動品がそろっている本。この形をゲームエンジンでやりたかった

前日との差分 = その日に学ぶこと。迷ったら前日に戻れる

1 Day の教材の形

docs/plans/DayXX.html + reference/DayXX

計画書

- 今日のゴール(動くもの1行)
- 事前に読む資料
- 理論の要点(図つき)
- 前 Day からの差分・設計書
- 完成条件(何が見えれば OK か)
- 改造課題(理解の確認)

実験台

計画書の中で、その日の理論をブラウザで動かして確かめる部品

例: 箱の接触点が何点で支えるか、TAA の履歴の混ぜ方、パーティクルの寿命 …

答えのコード

前 Day の完全コピー+その日の差分。**どの Day も単体で動く**

写経しても、改造しても、読むだけでもいい。1 Day の差分は数百行(1〜3時間)に収める

ポイント 「何を作るか」と「なぜそう動くか」と「答え」が、毎回そろって出てくる

教材の一例: Day 47 の計画書

PHASE 7 / DAY 47 / 計画書

摩擦・Sequential Impulses・スリープ(ミニ物理エンジン完成)

ロードマップへ戻る　「実験台」の印が付いたものは、このページで足した動かせる図

今日のゴール

事前に読む資料

理論の要点

前Dayからの差分概要

設計書

完成条件

計画書の中身

- 題材: 物理エンジンの摩擦・積み上げ・スリープ
- 理論の説明のすぐ下に、それを動かす実験台がある
- 例: 坂の箱が摩擦の強さで止まるか滑るかを、つまみで確かめる
- 実験台は答えの C# を JS に移したもの。同じ数値が出る

確かめる手順まで書く

AI に作らせていちばん怖いのは「それっぽいけど合っていない」。ビルド・検算・検証を、Claude Code のスキル(教材の作り方の手順書)に入れておく

1 Day の進め方

1

リファレンスを作る

「Day XX を作成して」の一言で、答えのコードと計画書ができる

create-day スキル

→

2

計画書（HTML）を見る

今日のゴール、理論の要点、図、実験台

docs/plans/DayXX.html

→

3

事前情報を読む

計画書が指す資料の章を読む

ゲームグラフィックス特論・西川本など

→

4

差分を見ながら写経

リファレンスと自分のコード（work）の差分をスクリプトで出し、差分が無くなるまで打つ

打ち終えたら動かして確かめる

自分のコードは1つを育てる

work は Day ごとに分けず、1つのプロジェクトに足していく。写経が終わったら、ロードマップの表に「済」と書く

差分が「今日の範囲」

どこを打てばいいかは差分が教えてくれる。1 Day は写経 1〜3 時間に収まる大きさ

教材を育てる: 途中で計画を変えていい

- **8/08** 発展的なグラフィックスを足した
  ディファード、レイトレーシングなど。「理論を学ぶ程度で」と頼んで教養編に
- **8/08** 物理エンジンを足した
  最初の計画には無かった。キャラが走り回るデモに要る
- **8/15** 卒業制作の題材を差し替えた
  敵が数百体の題材へ。スプライトバッチと ECS が「ゲームの必然」になる
- **9月** 膨らんだ Day を a/b に分けた
  44a/44b、54a/54b … 1 Day を写経 1〜3 時間に保つ
- **10月** 計画書を HTML に作り直した
  Markdown から、図と実験台のあるページへ
- **10月** VRM 編を足した
  トゥーン調のアバターを、仕様どおりに描いてみたくなった

自作教材のいちばんの利点

理解度や興味が変わったら、教材のほうを変えられる

分からなければ説明を足す、重ければ分ける、作りたくなったら章を足す。本ではできない

学び方 3/3

育った結果: 最終的なロードマップ

P0 Day 1
土台
ピクセルを 60fps

P1 Day 2〜10
ソフトラスタライザ
回転するモデル

P2 Day 11〜13
生 OpenGL
自作バインディングで三角形

P3 Day 14〜18
レンダラ
スプライト 1 万枚

P4 Day 19〜24
エンジンコア
ECS とシーン

P5 Day 25〜30
ゲーム
ゲーム 1 本完成

P6 Day 31〜40
現代の描画: 基礎
PBR・影・HDR でデモ v1

P7 Day 41〜51
3D ゲーム編
走り回れるデモ

P8 Day 52〜56
現代の描画: 応用
多光源・TAA・被写界深度

教養 Day 57〜67
最新の描画技術
レイトレ・Nanite の土台

VRM Day 68〜73
VRM 編
トゥーン調のアバター



Day 1　　帯の長さ = Day 数　　Day 73

57 Day → 73 Day

3D ゲーム編(物理・キャラ・エフェクト)と VRM 編を足した。a/b に分けた Day を数えると 87 個

題材も差し替えた

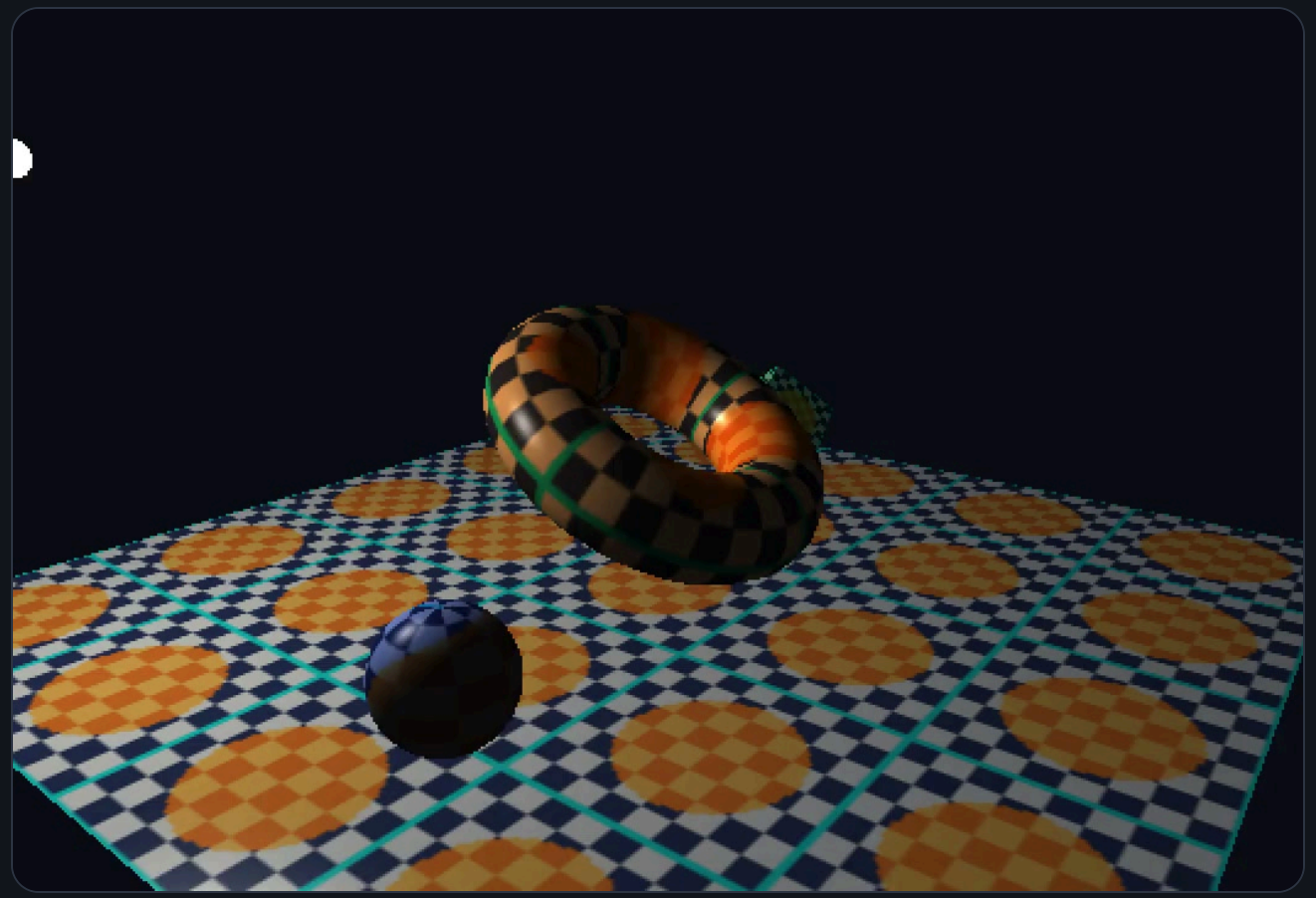
卒業制作はブロック崩しから、敵が数百体のアクションへ。スプライトバッチと ECS が効く題材に

Phase ごとに「動くもの」

各 Phase の終わりにマイルストーンを置く。Day 30 でゲーム1本、Day 56 で最終デモ

描画の土台: GPU の仕事を一度 CPU で書く

Day 1〜13



毎フレームの流れ

1	頂点を行列で変換(モデル→ビュー→投影)	Day 5〜6
2	w で割る(透視除算)と遠くが小さくなる	Day 6
3	三角形の内側を塗る(エッジ関数)	Day 3〜4
4	Z バッファで手前の物だけ残す	Day 7
5	テクスチャと陰影で色を決める	Day 8〜9
6	ピクセルの配列を画面へ転送	Day 1

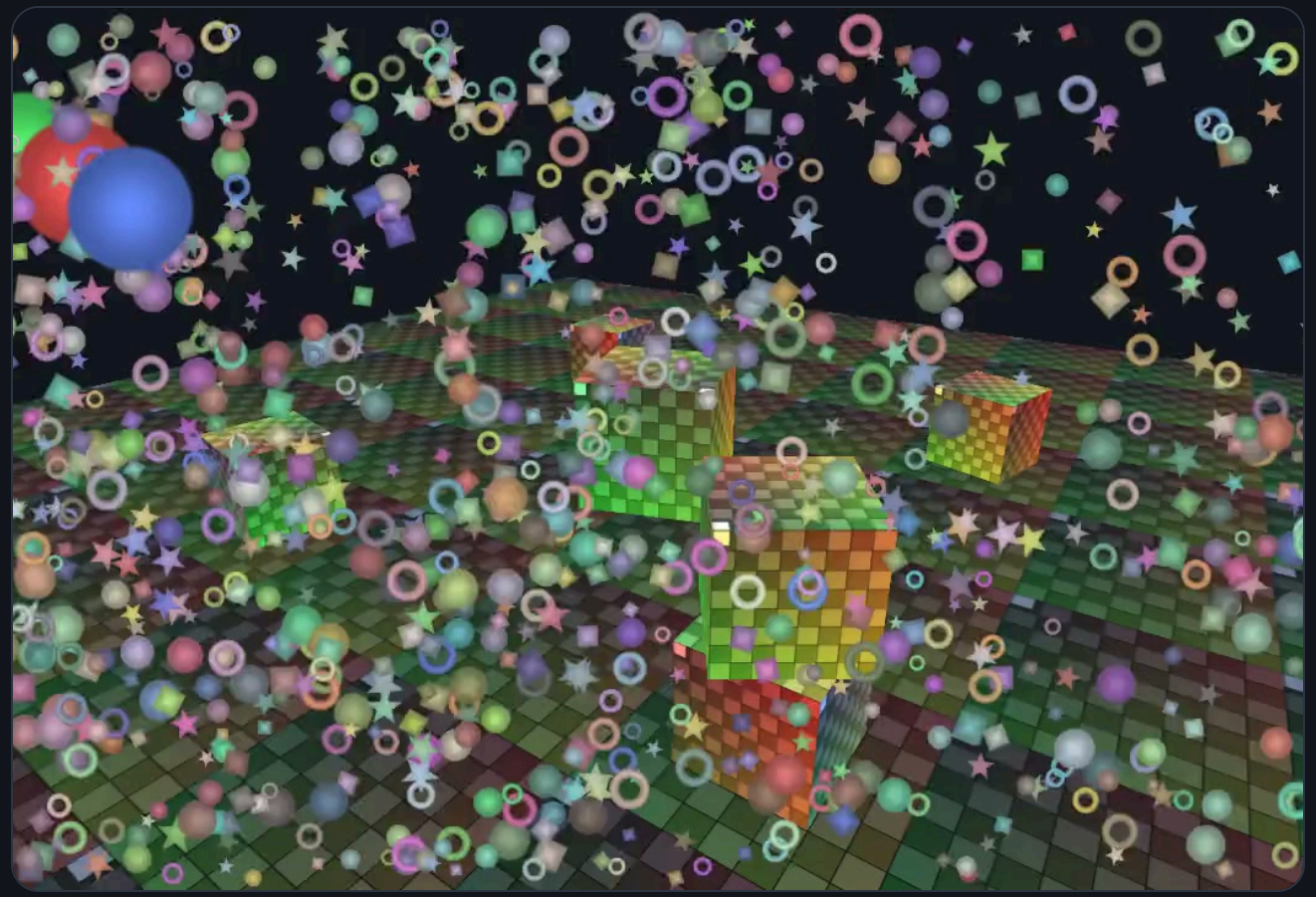
次のフレームへ

Day 11〜13: 同じことを GPU に頼む「儀式」も自前で書いた

分かったこと　ここを通すと、GPU の話が「知っていることの言い換え」になる

レンダラとエンジンコア

Day 14〜24



毎フレームの流れ

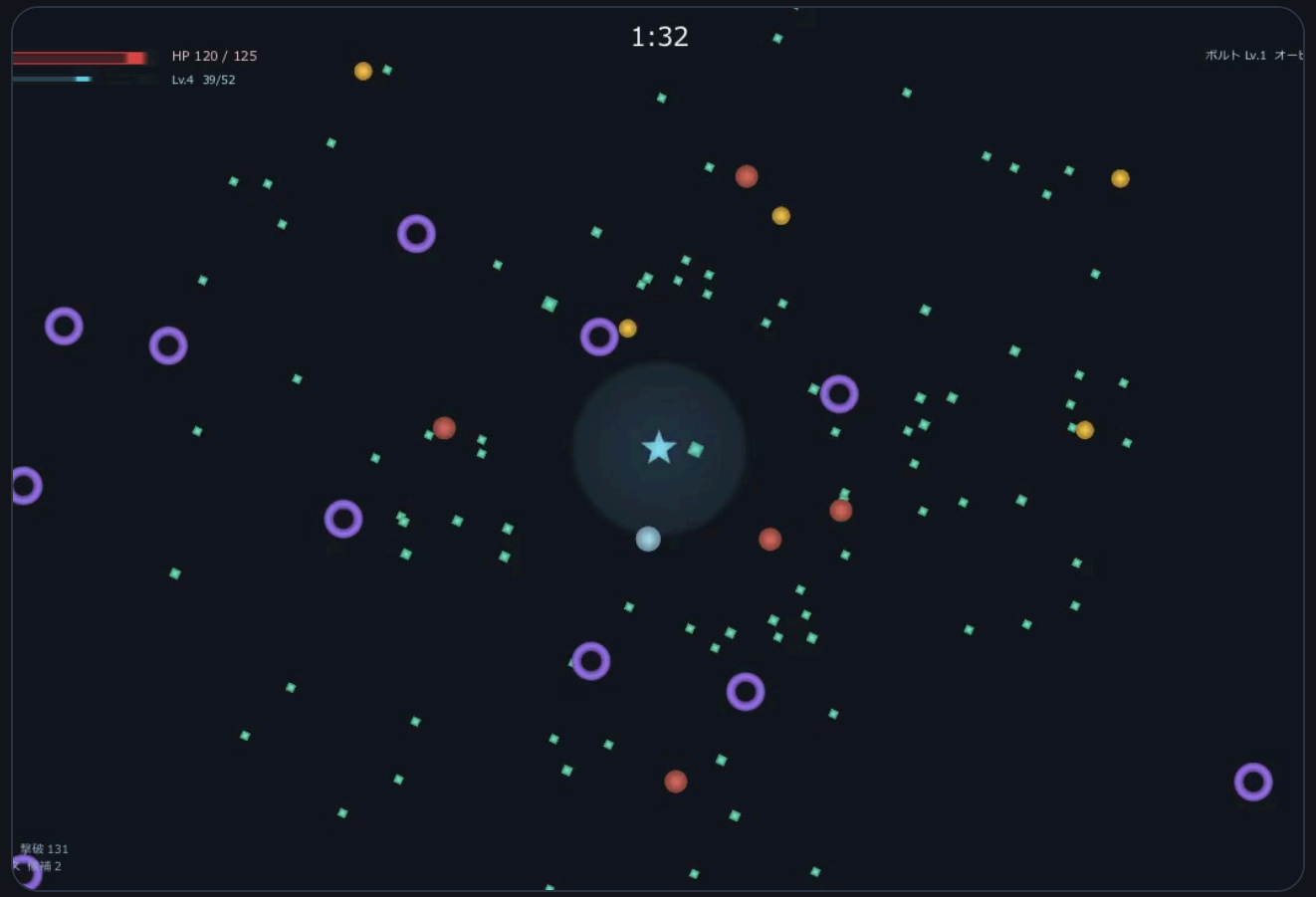
1	入力を集める（ポーリング+イベント）	Day 20
2	溜まった時間を 1/60 秒ずつ消化して更新	Day 19
3	エンティティを ECS で回す	Day 22→23
4	スプライトを溜めて、テクスチャ順に並べる	Day 17〜18
5	頂点バッファに詰めて、まとめて描く	Day 17
6	余った時間で補間して表示	Day 19

次のフレームへ

分かったこと　同じものを2回書く（GameObject → ECS）と、両方の設計の理由が腹に落ちる

卒業制作: 見下ろし型の時間耐久アクション

Day 25〜30



毎フレームの流れ

1. 入力でプレイヤーを動かす
2. 敵を湧かせる(数百体)
3. 武器が自動で攻撃する
4. 格子に登録して、近い組だけ候補にする — Day 26
5. 当たり判定(AABB・円・回転した箱) — Day 25
6. 経験値 → レベルアップで武器を選ぶ — Day 30

次のフレームへ(音・文字も自前: Day 27〜28)

分かったこと　敵が数百体の題材だから、1 万スプライトも ECS も「ゲームの必然」として効いた

成果物 4/9

HDR とポストエフェクト

Day 31, 37, 38



毎フレームの流れ（描いたあと）

1. シーンを HDR のバッファに描く（1.0 を超える明るさを残す） — Day 31
2. SSAO: 深度と法線から、隅の暗がりを作る — Day 37
3. ブルーム: 明るい所を抜き出し、ぼかして足す — Day 31
4. トーンマッピング（ACES）で画面に収める — Day 31
5. FXAA でギザギザを消し、色を整える — Day 38

次のフレームへ

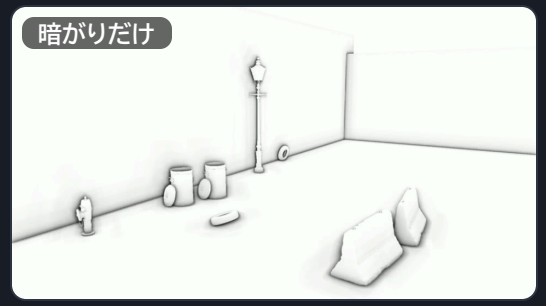
分かったこと 「映画っぽさ」の正体は、描いたあとの後処理

成果物 4/9 を1つずつ

後処理は、それぞれ何をしているか

Day 31, 37, 38

SSAO 物と物が接する所に暗がりを作る





ブルーム 明るい所を抜き出し、ぼかして足す



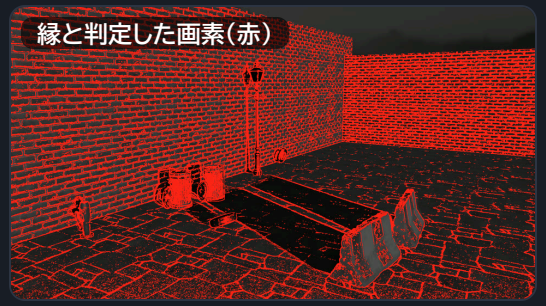


トーンマッピング HDR の明るさを画面の 0〜1 に収める





FXAA 画面全体を見て、縁の画素だけぼかす





材質と光: PBR・IBL・影

Day 32〜36, 39〜40



毎フレームの流れ

1. (起動時)glTF からモデルと材質を読む — Day 32
2. 光から見た深度 = 影の地図を描く — Day 33
3. 法線マップで面の向きを細かくする — Day 34
4. 金属・粗さで反射を計算(Cook-Torrance) — Day 35
5. HDRI から環境光を足す(IBL) — Day 36
6. 影は周りを何点か見てぼかす(PCF) — Day 33

次のフレームへ(このあと後処理)

分かったこと　HDRI 1 枚で、シーンの説得力が激変する(費用対効果が最大)

成果物 5/9 を1つずつ

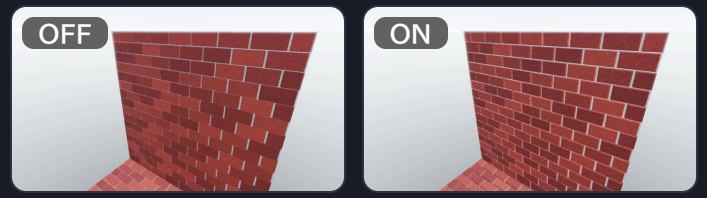
材質と光は、それぞれ何をしているか

Day 33~36

② 影 光から見た深度で、影を落とす



③ 法線マップ 平らな面に凹凸の向きを与える



④ PBR 金属度と粗さで反射を決める



⑤ IBL 周りの景色を光として使う

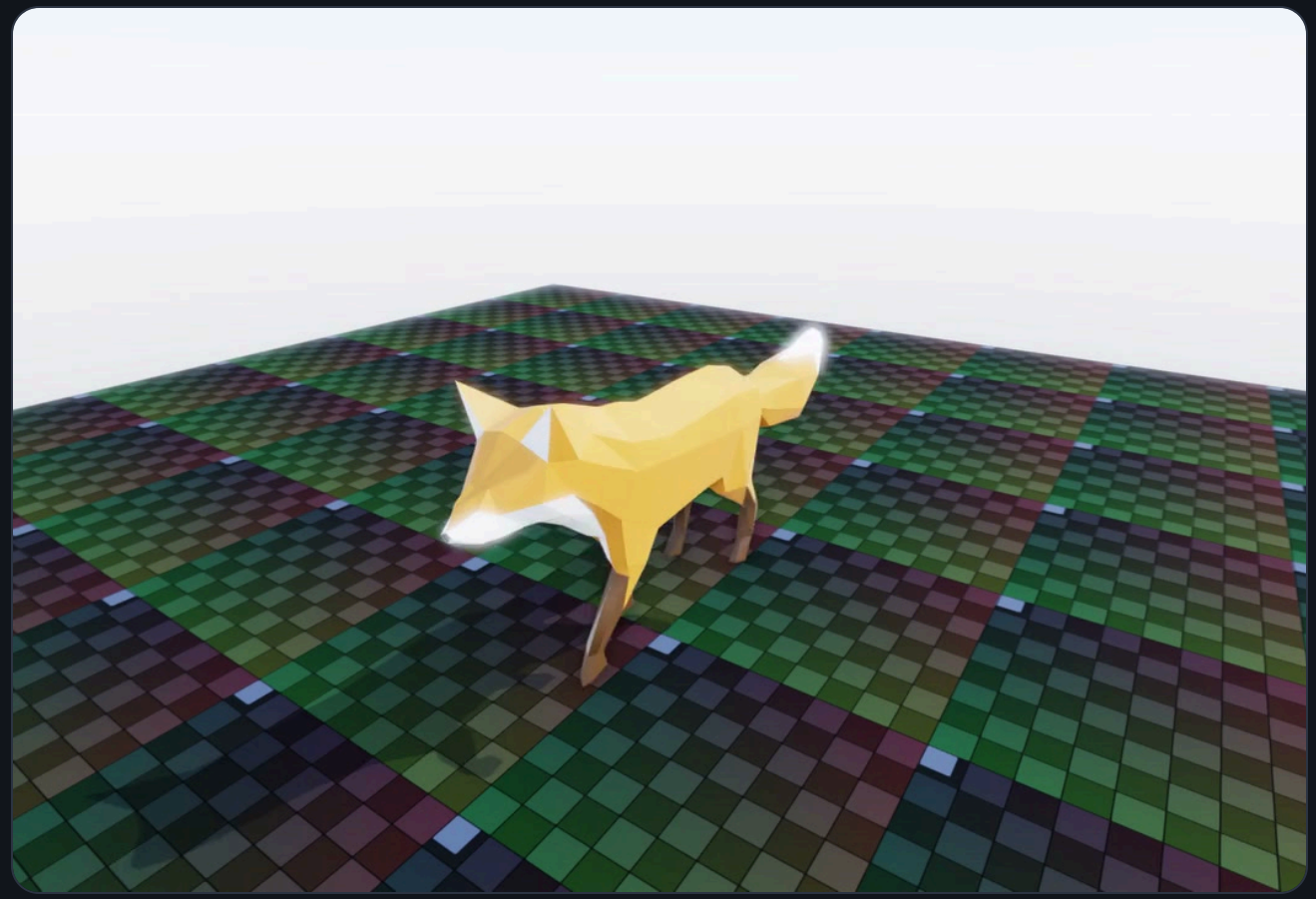


⑥ PCF 影の縁を何点か見てぼかす



キャラクター: アニメーションと移動

Day 41~42, 45b



毎フレームの流れ

1. 移動速度から idle / walk / run の重みを決める — Day 42
2. 各クリップを時刻で取り出して混ぜる — Day 42
3. ボーンの行列を親から順に掛ける — Day 41
4. 頂点シェーダで頂点をボーンに追従させる — Day 41
5. 体はカプセル。坂・段差に沿って滑らせ、接地を判定 — Day 45b

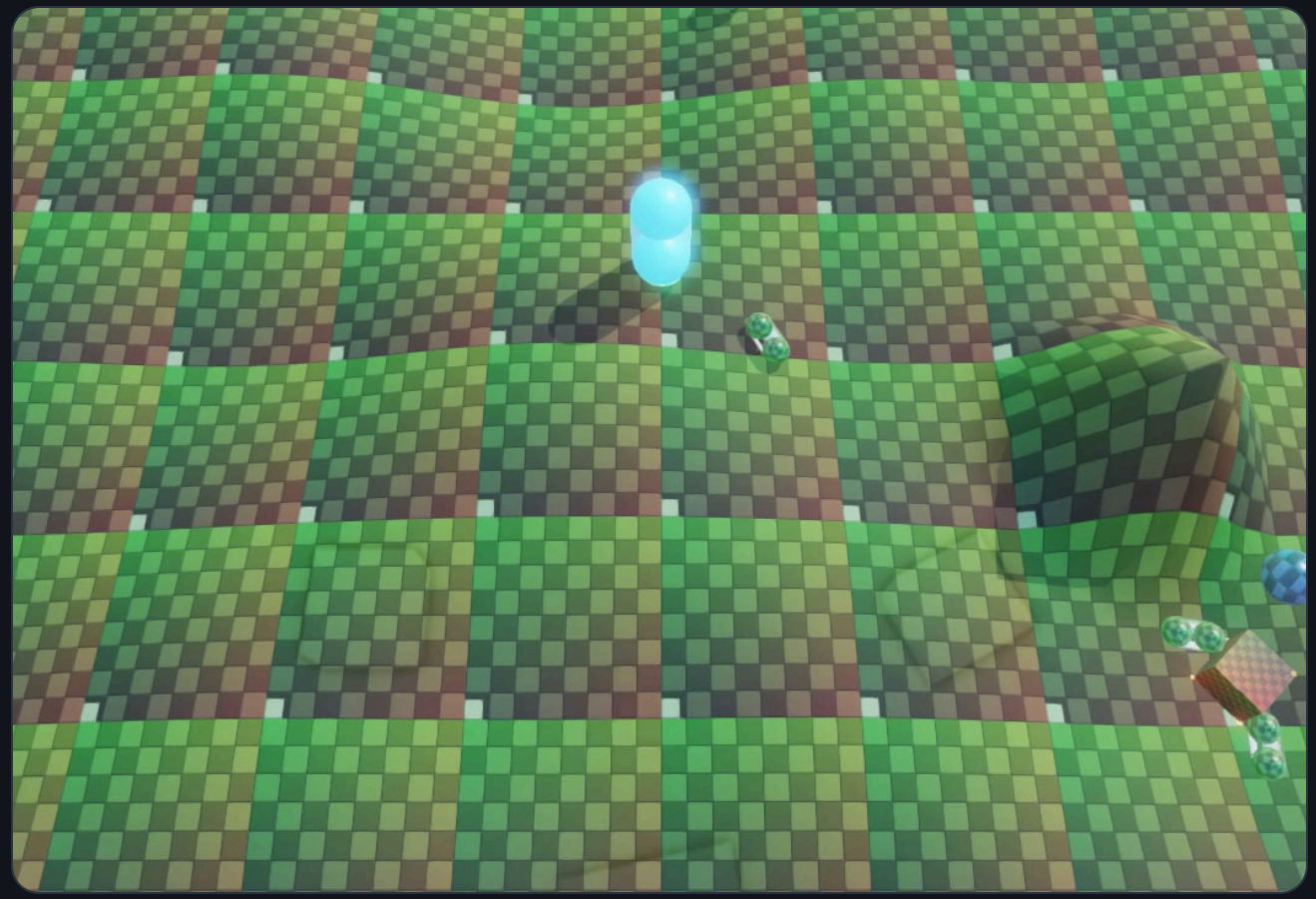
次のフレームへ

分かったこと　キャラは剛体として転がさない。「自分で動かすカプセル」が 3D ゲーム実用の要

成果物 7/9

ミニ物理エンジン

Day 43~47



毎フレームの流れ

1	重力などの力から速度を進める	Day 43
2	格子で、近い組だけ候補にする	Day 46b
3	接触判定: 球・箱・カプセル・地形	Day 43~46a
4	接触点をまとめる（1組最大 4 点）	Day 44a
5	重なりと速度を解く（摩擦・反発、8 周）	Day 47
6	新しい速度で位置を進める	Day 43
7	止まった山は眠らせる	Day 47

次のフレームへ

分かったこと　接触点を 1 点ずつ順に解くと、床の箱が揺れ止まない。積むと「4 点を同時に解く」意味が分かる

エフェクトとプレイアブルデモ

Day 48〜51



1 フレームの全体（ここまでの全部）

1	入力 → 三人称カメラ	Day 20
2	キャラを動かす（カプセル）	Day 45b
3	アニメを混ぜる	Day 42
4	物理を 1 ステップ進める	Day 47
5	足元の土埃・ヒット・軌跡	Day 49〜50
6	描画（PBR・影・IBL）	Day 32〜36
7	後処理（SSAO・ブルーム・AA）	Day 31〜38

次のフレームへ

分かったこと　パーティクルは深度で薄めて、地面への「刺さり」を消すのが見栄えのコツ

仕上げ: 多光源・TAA・被写界深度

Day 52~56



毎フレームの流れ

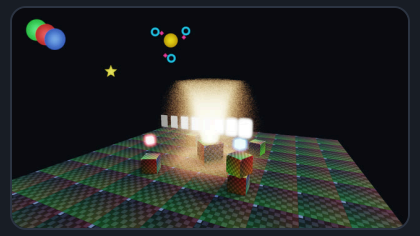
1. G-Buffer に位置・法線・材質を書き出す — Day 52a
2. 点光源 1024 個を「光の球」で足す — Day 52b
3. Forward+ と比べる（光を升目に配る） — Day 53
4. TAA: 毎フレーム少しずらし、履歴と混ぜる — Day 54
5. 被写界深度・モーションブラー — Day 55

次のフレームへ

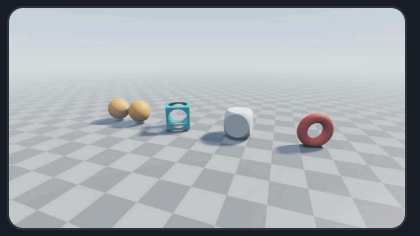
分かったこと　TAA はカメラだけが動く絵には完全に効く。動く物は、その速度で履歴を追わないと輪郭がにじむ

その先: 教養編(1)

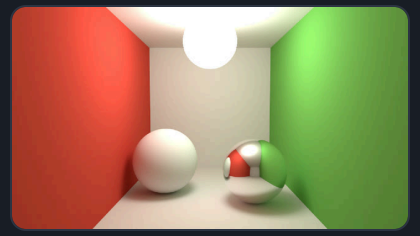
Day 57~63



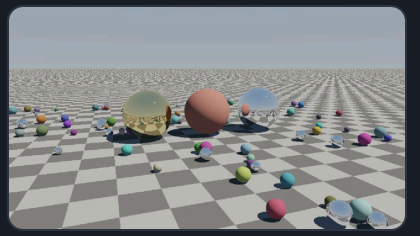
Day 57
コンピュートシェーダ
描くためではなく、計算のために GPU を使う仕組み。100 万粒の位置と速度を GPU に置いたまま、毎フレーム並列に動かす。同じことを CPU でやると 20 倍以上遅い



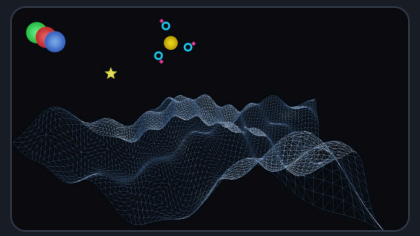
Day 58
レイマーチング
ポリゴンを使わず、「形までの距離」を返す関数だけで描く。画素ごとに光線を飛ばし、距離の分だけ進むのを繰り返して表面を見つける



Day 59~61
パストレーサ
光が跳ね返る道筋を乱数でたくさん追い、写真のような光の回り込みを出す(映画の CG と同じ考え方)。CPU で書いて理屈を確かめ、GPU に移して約 30 倍速くする



Day 62
ハードウェア RT
最近の GPU にある、レイトレーシング専用の回路を使う。物体を箱の木で包んだ「加速構造」を Vulkan で作り、光線と物体の当たり判定を GPU に任せる



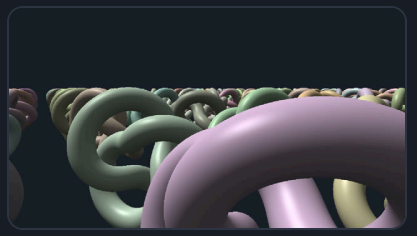
Day 63
ジオメトリ・テッセレーション
粗い形だけを送り、GPU に三角形を作り直させる段。テッセレーションは細かく割る段で、手前は細かく、奥は粗くできる。写真は 256 個の点から 8,192 枚に割った地形

その先: 教養編(2)

Day 64~67



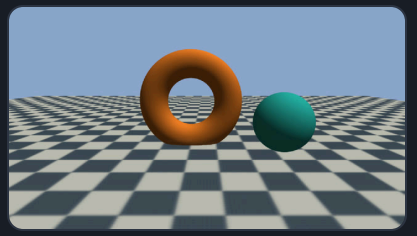
Day 64
メッシュシェーダ
モデルを「メッシュレット」(三角形 100 枚ほどの小さな塊)に切り、塊ごとに処理する新しい描画の流れ。画面の外や裏向きの塊を、GPU が自分で捨てられる。写真は塊ごとに色を変えたもの



Day 65
GPU 駆動・Hi-Z
「何を描くか」の判断を CPU ではなく GPU にさせる。深度を段々に縮めた Hi-Z を使い、手前の物に隠れた物を描く前に捨てる。UE5 の Nanite の土台になっている考え方



Day 66
焼いたプローブ GI
壁や床で跳ね返った光が、ほかの物を照らす効果(GI)。空間に格子状に測る点(プローブ)を置き、届く照り返しを前もって計算して(焼いて)おく。Lumen や DDGI は、これを毎フレーム更新する発展形



Day 67
Gaussian Splatting
写真から 3D の場面を復元する、新しい表し方。ポリゴンの代わりに、色と透けぐあいを持つぼやけた楕円体を、数十万個重ねて描く。写真の床も球も、すべて楕円体の集まり

ねらい　UE5 の Nanite / Lumen が「何を解決しているか」が分かるところまで

その先: VRM 編

Day 68〜73



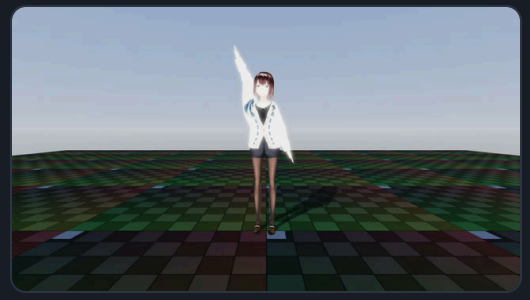
Day 68
VRM と MToon
トゥーン調のキャラを裏通りに立たせる



Day 69
表情
モーフを混ぜて笑顔に。まばたきも自動



Day 70
揺れもの
髪が遅れてついてくる(線は当たり判定)



Day 71
アニメーション
手を振るクリップを、アバターの骨に写す



Day 72
セルルックの仕上げ
左が仕上げ前、右が仕上げ後の色



Day 73
VRM で遊ぶ
歩き・走りに合わせた足運びと、エモート

考察

やってみて、出来たことと出来ていないこと

メリット・成果

- ゲームエンジンの作りとレンダリングパイプライン、キャラクターが動く仕組みなどの大まかな理解が出来た

  ゲームループ、ECS、シーン。頂点の変換 → 塗る → 陰影 → 後処理の流れ。骨を動かし、GPU が頂点を曲げる（スキニング）

- グラフィックスエンジニアの話が、よく分かるようになった（用語と仕組みがつながった）
- 業務でも、もう活きている（GPU の描画結果を CPU に読み戻す処理で、学習の知識が役に立った）

デメリット・出来ていないこと

- コードもドキュメントも無駄が多くなりがち（同じような処理や説明、念のための処理が増える）
- 見た目の検証がまだ甘い（画面が正しいかは人の目に頼る部分が残る）
- 正しさの保証が無い（書籍のような校正が無く、誤りを学ぶ側が見抜けない）
- 技術を組み合わせて絵を作る力は、まだ身についていない（良い絵かの判断は AI にも難しい）

まとめ

自分に合った教材は、作れる

独学の効率が上がった

自分にとって都合の良い教材を作ったことで、学習の効率が大きく上がったと実感している

約 2.5 か月で Day 1〜51 まで進められた

作るときのコツ

条件を渡す

言語・レイヤ・順番・ペース・ゴールを伝えると、その人専用のカリキュラムになる

確かめる手順を書く

スキルに確かめる手順まで書くと、計画書を 88 枚作っても品質がそろう

途中で変えていい

知りたい内容が増えたら足す、重ければ分ける。教材が自分に合わせて育つ

ひとこと　これは私が試したやり方の一例。みなさんも、それぞれの条件を伝えて、自分に合った教材を作ってみてください

部内発表での質疑から

Q 理解が難しいときは、どうした？

A 事前情報の理解が難しければ、諦めて写経に進む

「完璧より前進」

Q リファレンスの修正の指示は出した？

A
- あまり手直しはさせていない
- 強いて言うと、サイズが大きいものを分割させたぐらい（44a / 44b など）
- 理解できなかったら、その場で聞いていた
- たまに Claude Code 自身がミスを見つけて、何日分か遡って直すことはあった

ありがとうございました

質問どうぞ